CS543 · WEB SERVICES · ASSIGNMENT 09 OF 12

Securing the API

Goal. Take the REST service you built for Assignment 8 and **secure it**: add a login that issues a **JWT**, a middleware that authenticates every request, and a **role rule** that controls who may change data. Replicate exactly the service shown in the tutorial — do not introduce a new one.

TASK

Reuse your existing service (e.g. **Catalogue** — `GET /items`, `POST /items`). Add two users with different roles, a login, token verification, and a role check. Follow the Securing-the-API tutorial as your template.

#	Do this
1	**Users & hashing** — Seed at least two users with different **roles** (e.g. `customer` and `restaurant`). Store each password as a **bcrypt hash**, never in plain text.
2	**Login** — Add `POST /login`. Verify the password with `bcrypt.compareSync`; on success return a **JWT** signed with your secret, carrying the user id and role, with an **expiry** (`expiresIn`). On failure reply **401**.
3	**Authentication** — Write an `auth` middleware that reads the `Authorization: Bearer <token>` header, verifies it with `jwt.verify`, and replies **401** if the token is missing, invalid, or expired. Protect your read route(s) with it.
4	**Authorization** — Write a `requireRole(...)` middleware. Protect your **write** route (POST/PUT/DELETE) so only the privileged role may change data — reply **403** for any other authenticated user.
5	**Secrets & config** — Keep the JWT secret out of your code: read it from `process.env.JWT_SECRET` (a `.env` file, git-ignored). Commit a `.env.example` with a blank value.
6	**Demonstrate** — Using curl or the browser, capture: a request with **no token → 401**; a successful **login → token**; a request **with the token → 200**; the privileged role **→ 201/200**; another role on the write route **→ 403**.

THE SHAPE OF THE TWO GUARDS

```
authenticate, then authorize — in that order
function auth(req, res, next) {                        // who are you?
  const token = (req.get('Authorization')||'').split(' ')[1];
  try { req.user = jwt.verify(token, SECRET); next(); }
  catch { res.status(401).json({ error: 'invalid token' }); }
}
const requireRole = (...ok) => (req, res, next) =>     // may you?
  ok.includes(req.user.role) ? next() : res.status(403).json({error:'forbidden'});

app.post('/items', auth, requireRole('restaurant','admin'), handler);
```

DELIVERABLES

- `server.js` — your secured service: `POST /login`, the `auth` and `requireRole` middleware, and your routes behind them.
- `package.json` — with `express`, `jsonwebtoken`, and `bcryptjs` listed, plus a `.env.example` (and `.env` git-ignored).
- A **GitHub repo** with the above + a **Google Drive link** with a short demo video and screenshots of the five cases from Task 6 (401 / login / 200 / 201 / 403).
- One submission per team via Google Classroom — repo link, with roll numbers and names on the README.

ANSWER IN THE README

1. Which route(s) require only authentication, and which also require a role? Give the method + path and the role allowed for each.

2. Show one `401` and one `403` from your demo. In one line each, say why the server returned that code (not the other).

3. What is inside your JWT payload? Why is it safe to trust the `role` claim on the server, and why must the secret stay out of your code?